赛题 11：Code Agent 上下文治理
算法、真实模型评测与资源研究报告

本地项目研究记录

2026 年 10 月 11 日；研究与验证报告

摘要

本项目研究在代码生成质量不下降的前提下，缩短上下文与首 Token 时延。实现包含完整 Python AST 切块、保守跨文件依赖闭包、用户原文保护、查询相关度与历史效用排序、日志抽取、实际模型 Token 预算和独立功能评测。在 14 个自建任务、3 轮成对真实生成中，1.5B 模型完整/压缩上下文均通过 9 题，3B 模型均通过 10 题。扩充至 20 题后，3B 双调用复核完整 13 题、压缩 16 题，无成对或初稿到修订退化；加入公开源码静态提示仍为相同正确率，保留为负结果。双调用累计输入 Token 加权减少 87.01%，第一稿端到端首 Token 逐题改善均值 87.80%，到最终代码阶段时间改善均值 26.07%。仍有 4 题错误，不能将“不劣于基线”等同于“全部代码正确”。后续 7B NF4 单调用在相同 20 题上通过 18 题，但首次测量的新增 6 题仅通过 2 题，因此不能由原集合的改善推定工程泛化。新增会话缓存在 23 个工作负载上保持相同选择，温热请求中位加速 2.64 倍；46 种编辑场景增量索引中位加速 2.44 倍，初始化另计。TypeScript 编译器基座新增具名类型闭包与编辑复用，实际编译执行检查通过；三阶段模型批评在 26 题上出现两项成对退化，新 4 个工作流首次单调用均失败，保留负结果。30 题公开样例反馈两轮完整/压缩通过 19/20 题，未提升压缩组正确数；按层 KV 卸载配合分块回收完成 32688 输入与 13 输出，单对完整 greedy Token 一致，资源与时延证据另列。更大工程任务、DevEco/ArkTS 集成与未知官方数据尚未完成。本报告保留失败、口径与资源边界。

1 赛题介绍与实现范围

官方要求 [1] 包括语义、新颖性与历史效用等重要性因素，单次分析在 100ms 内完成；根据模型窗口和任务需求选择保留、摘要或去重，并记录每次压缩与 session 指标。代码生成质量不劣化是计分前提，压缩与 TTFT 权重为 60% 和 40%，理想改善分别为 25%。本地按 `spec/metadata/repo_bundles/templates` 组织输入，没有官方工程包时以 SHA-256 固定快照，并将 `base_commit` 明确设为 null。

当前治理核心仅依赖 Python 标准库；模型评测环境独立安装 Transformers 并使用本机 CUDA PyTorch。质量测试覆盖本地 Python 纯函数与多文件修复。另已实现 TypeScript Compiler API 适配与编译执行验收，但 DevEco 产品内部注入和完整 ArkTS 专用语法/SDK 适配尚未完成。词项相关度属于轻量检索，并非向量语义模型；抽取式日志压缩不生成新事实。正式质量指标、官方模型和平台时延统计尚不能由自建 fixture 推定。

2 经典与相关工作

2.1 提示压缩与长上下文利用

LLMLingua[2] 以预算控制和迭代 Token 压缩减少提示，LongLLMLingua[3] 进一步考虑问题相关信息密度及位置信息。LLMLingua-2[4] 用数据蒸馏训练 Token 分类器，提升通用压缩效率。这些工作为“按任务保留有用信息”提供方法基础，但它们的文本压缩结果不能直接保证代码语法、符号引用或跨文件依赖完整。因此本项目首先保持整个函数、导入和必要配置闭包，不将逐 Token 删除直接用于工程源码。

Lost in the Middle[5] 展示了长上下文检索中的位置敏感性。这支持减少无关材料的研究动机，不证明任意删减都能改善代码生成。RepoCoder[6] 通过迭代仓库检索与生成利用跨文件信息；本项目采用静态闭包提供确定的最低依赖保护，后续可在该安全边界之外加入迭代检索。BM25[7] 考虑词频饱和、文档长度与逆文档频率；当前实现只是归一化词项重叠，不应把它称为 BM25。短标识符拆分、文档频率和依赖组成本排序仍有优化余量。

2.2 推理系统与质量评估

GQA[8] 用少于查询头数量的 KV 头减少缓存；FlashAttention[9] 通过 IO 感知的精确注意力计算减少中间矩阵物化；PagedAttention[10] 处理服务中 KV 缓存的分页与共享。这些属于推理系统优化，与删减提示是不同层面的收益来源。本机 Windows PyTorch 构建未包含 FlashAttention 内核，原生 GQA 走高内存数学实现。本项目只在注意力适配层扩展 K/V 到查询头数，允许已安装普通 SDPA 内核自行分派 [17, 18]，没有实现 FlashAttention 或 PagedAttention，也没有重新训练 GQA 权重。

Self-Refine[11] 研究用同一模型反馈并修订初稿。CodeT[12] 由模型生成测试，使用执行与候选间一致性选择代码；Self-Debugging[13] 研究执行结果与代码解释辅助自我修复。本项目普通复核只提供用户要求、可见上下文和草稿；另有可选公开样例执行反馈，其预期值是可能错误的模型输出，不是隐藏 oracle。本实现没有 CodeT 的多候选一致性排名，也没有复现这些论文的官方 benchmark 或训练方法。隐藏功能测试在每条完整流程的全部模型调用结束后评分，不进入该流程或后续独立流程的提示；不是等整批全部任务结束才评分。公开样例生成不看初稿，只看公开要求/上下文；返回至多 6 项 JSON 记录，参数和预期结果经受限 AST 字面量检查。初代 v17 函数名称限原目标文件公开函数，拒绝任意调用表达式、超深/超大数据和无限数值。这一 v17 契约假设目标函数已存在。初代 14 题的目标文件只有注释占位，因此即使模型返回合法 JSON，函数作用域检查也会拒绝其新建函数；此兼容性缺陷属于本地反馈基座，不作为模型正确率结论。v21 隔离候选从公开任务开头的 `Implement f(args)` 声明补足允许函数，显式使用 Python 列表字面量表示位置参数，避免单参数元组遗漏逗号，生成规模缩至 1–3 例。没有解析隐藏测试或答案；支持范围是该英文公开声明和已有顶层函数，尚不覆盖任意自然语言 API 描述、方法或 kwargs。8 项针对性检查通过（20.20 秒），含 4 项旧协议安全/执行检查以及占位、新参数容器、越界调用拒绝与旧提示一致性。独立审查随后发现旧 helper 允许名单与提示不一致，以及 Python 关键字函数名可能造成模板 SyntaxError。v22 另立冻结版本，让明确声明的必需 API 优先于旧 helper，并拒绝关键字；提示和解析器共用同一名单。10 项检查全部通过（12.94 秒），包含旧 4 项、原修订 4 项及两项新边界；旧 `literal-v17` 提示保持逐字一致。这些检查证明修复的兼容行为，不证明模型例子的预期答案或工

程代码质量；实际模型结论另批评测。可信 unittest 模板在既有受检进程内执行初稿，反馈仅含样例和稳定的成功/失败状态，无隐藏参数/真值。4 项新增测试验证拒绝与稳定重放路径；字面量经 AST 规范化、补齐注释与 Unicode 边界后重新通过，耗时 15.04 秒。质量批次与额外生成、执行成本在后文单列。SWE-bench[16] 以真实仓库问题及补丁评测工程能力；当前小型 Python 集合远小于该类工作负载，不能换算成 SWE-bench 成绩。

3 当前算法与工程构思

3.1 快照、源码与依赖保护

快照 hash 包括路径和文件字节，加载时重算，避免更改测试对象而不留记录。按完整顶层 AST 区间切分 Python 源码，保留装饰器、导入、空白/注释和分号合并区间。建立模块全局符号到定义块的索引，解析普通、相对、包及星号导入，保守追踪跨文件引用与顶层控制流中的绑定。一次后续审计补齐累加赋值、循环/海象表达式绑定、属性和下标写入，以及装饰器、默认实参和模块初始化调用的影响；不把函数局部变量当模块导出。不能精确解析的模块初始化块保守加入闭包，可能扩大预算。5 项新增测试直接比较完整模块与选择后模块执行语义；现有 26 题的依赖图修复前后完全相同。历史基础环境核心回归 48 项，47 项通过、1 项需可选模型依赖而跳过，耗时 100.11 秒。最终 v24 推广后在模型环境执行全部 48 项，全部通过、无跳过，测试计时 157.67 秒，含解释器启动等的流程墙时 161.14 秒；属于 CPU 功能回归，不是新模型优化批次。之后 TypeScript 增量 worker 的相同 5 项编译/执行语义检查单独通过，耗时 11.26 秒。动态 import、反射和 eval 未完整覆盖；依赖不确定时应扩大保护或回退完整上下文。

保护集合包括原始用户要求、目标源码、调用者指定资料和当前错误；其传递闭包一起保护。闭包存在环时用访问集合终止。保护集合超预算时显式报错，不截断约束、函数或依赖。原 20 题中后增的六个多文件任务仅固定目标文件，辅助函数及配置由已存在导入拉入闭包。再增的六个结构任务中，稳定拓扑、最短分层及稀疏乘法三题也含跨文件 helper 导入，另三题无此依赖；原 14 题大部分辅助文件直接固定，不能用它们单独宣称非保护依赖召回能力。

3.2 重要性、去重与预算

对每个非保护块 x，当前基础重要性为

$$s(x) = \frac{|Q \cap \mathrm{terms}(x)|}{\max(1, |Q|)} + \mathrm{clip}(u_x, 0, 1) - d_x.$$

Q 移除通用任务词与文件扩展名；标识符保留完整形式并拆分下划线、点号和 camelCase，中文目前以单字词项处理。u_x 为调用者提供的历史效用；$d_x = 1$ 仅用于重复工具/记忆文本的排序惩罚，后续还有精确重复检查。当前没有由长期真实成功率自动学习 u_x，也没有近似语义重复检测。

非正相关源码、工具和记忆可丢弃，需求仍保留；相同源码文本但不同定义身份不能误作重复。按重要性稳定排序，完整依赖组原子加入。首先批量计算可能入选块的 Token 成本，以总成本筛除明显放不下的组；每个实际接受的组合仍按原始来源顺序完整分词，以处理 Token 边界合并和排列变化。成本估算可能错拒绝少数能装下的组，但不能放入超预算组。超长工具输出仅抽取查询相关行或错误/差异/警告行，不合成摘要。

生产请求不对全部被丢弃上下文再次分词以生成诊断字段；完整输入 Token 在独立报告阶段测量。这一口径必须和“带完整诊断”的治理时间分开。每次实际 chat prompt 仍由真实模型 Tokenizer 计数。`PreparedContext` 缓存不可变上下文词项、依赖闭包和候选成本；源内容/依赖变更拒绝旧索引，计数器实例变化清空成本，接受预算始终重新计数。缓存由调用会话持有，无全局原文缓存；初始化时间不能藏进温热请求时延之外而不报告。

3.3 TypeScript 源码适配与类型依赖精简

可选 Node 常驻 worker 使用锁定的 TypeScript 6.0.3 Compiler API[14]，对调用者提供的虚拟工程建立 AST、符号和跨文件引用，不执行输入源码，不访问网络获取依赖。保留完整声明及原始 UTF-16 区间对应的注释/Unicode 文本；目标文件、模块初始化及普通运行时导入保守保护。动态或缺失相对导入、eval 和解析失败回退整个已提供工程。`.ets` 扩展名可接收，但专用 `struct` 语法失败时仅全量保留，不能宣称已有 ArkTS 语义编译器。外部 SDK 或模块别名只给出未解析警告，必须由产品工程补齐。

类型专用导入/导出在 JavaScript 中擦除 [15]。新策略仅对具名类型导入/导出，且目标模块没有初始化、变量、类等执行块时按真实符号及继承引用闭包保留；命名空间、混合导入或含执行副作用的模块继续全模块保护。这一保守边界避免用类型裁剪错误删除运行时初始化。5 项适配测试包含真实 `tsc` 编译与 Node 执行、跨 barrel 与累加配置、Unicode/CRLF、修改 helper 后缓存失效、动态/ArkTS 回退，以及 500 个无关接口精简后的编译。它们验证源码处理，不代表模型已能生成 TypeScript 工程补丁。

3.4 模型适配与会话指标

本地生成使用官方 Qwen2.5-Coder 0.5B、1.5B 及 3B Instruct 权重 [21, 22, 23]，逐文件 commit、大小和 SHA-256 锁定，禁止远程 Python 代码执行，加载前核验。0.5B 与 1.5B 为 Apache-2.0，3B 为 Qwen research license，许可文件保留，不能混称 Apache-2.0。CUDA 使用 FP16、greedy、无采样；改变注意力计算内核可能有浮点差异，功能质量另测。

3B 通过 1024-Token 分块预填充降低激活峰值。所有提示 Token 按原始顺序进入同一 DynamicCache，最终非空片段交给 generate；位置与 causal mask 保持连续，不是裁切上下文。小型随机 CPU 模型测试证明首个生成 Token logits 在容差内与不分块一致，并得到相同 greedy 输出；这不证明 GPU 不同内核的逐位相同。

可选 NF4 后端使用 bitsandbytes 的 4 位线性层、双重量化与 FP16 计算，以降低较强模型的权重内存 [27, 24, 25]；NF4 与双重量化来自 QLoRA 工作，本项目只使用其量化推理基础，没有进行 LoRA 训练。这会改变数值精度，不能把 NF4 较大模型与 FP16 较小模型的质量差异归因于纯规模。本机 64 乘 64 CUDA 层成功后完成整个 7B 模型单调用评测，质量与整体峰值如下单列。依赖固定为 bitsandbytes 0.50.2 与 accelerate 1.15.0。

SQLite WAL 保存每 session、每 request 的实际 Token、成对首 Token、功能结果、治理耗时和 provenance。会话压缩采用 $1 - \sum C_i / \sum B_i$，不能用各请求比例均值替代；会话 TTFT 降低率为 $\mathrm{median}_i(1 - C_i^{\mathrm{e2e}}/B_i^{\mathrm{e2e}})$，不是两组 TTFT 中位数之比；研究批次另列逐任务降低率均值。未知质量不是成功，重复 request ID 报错，事务结束后关闭连接。

4 测试构造与可信评测

最初 6 题覆盖范围、分块、稳定去重、slug、配置计费和 CSV 转换；后增 8 题覆盖连续游程、旋转、区间合并、辅助验证、直方图、Unicode 前缀、窗口和配置阈值。每题包含固定边界与随机/变形性质，测试和 oracle 不在模型快照中。生成只替换目标文件，非目标文件字节必须不变。可信驱动记录实际执行的测试数量，跳过、提前退出、伪造完成、污染测试框架不能视为通过。AST 门槛针对本地纯函数，禁止进程/框架反射、危险动态执行及属性写入；它不是通用 Python 安全沙箱，也不能用于类对象写入和任意文件 IO 任务。新增工作流需要标准库深拷贝，后续纯函数门槛加入 `copy/heapq`；改动前 1088 份已存输出的静态审计没有使用这两模块。离线重放按原报告起始 SHA 选择冻结的质量门槛，避免未来库策略变化重写历史评分。2 项新增检查确认深拷贝/堆排序执行及错误门槛 SHA 拒绝，耗时 1.91 秒；不同质量门槛版本仍应声明，不能作为压缩算法收益。

新 6 题覆盖价格与校验、行解析及配置链、截断均值、记录选择、谓词分区和键归一化汇总。既有导出必须保持，辅助函数通过 monkeypatch 性质测试确认被实际调用，不能只凭导入语句判定依赖使用。新的 20 题仍是自建集合，原 14 题已反复用于开发，不再称为未见持出集。新 6 题首次评测前固定，后续如据结果调优也应归为开发数据。

模型评测按题和轮次交替完整/压缩顺序，排除模型载入与 warmup。另在模型首次测量前构造 4 个工作流任务：事务编辑、嵌套配置合并、原坐标文本补丁、TTL/LRU 缓存轨迹。其 16 个测试方法涵盖失败原子性、别名/深拷贝、冲突坐标、过期与淘汰、公开 helper 调用。最初 v13 的 oracle 审计发现测试 monkeypatch 恢复后，已重载 solution 仍持有旧 helper 别名；创建 v13b 修复测试清理并重新载入，4 个参考实现全部通过、占位实现全部拒绝。这一测试装置错误发生在任何模型测量前，保留失败记录，没有计作算法收益。还对 26 个旧任务及 4 个新任务只重组压缩后保留的源码模块，用独立参考目标替换占位，30 个任务全部通过隐藏测试；整批 143.52 秒。该审计验证这些快照的依赖完整性，不证明任意动态工程可靠。首 Token 回调同步 CUDA，从编码/传输/预填充到第一个生成 Token。端到端首 Token 加上每次治理和请求构造；加载、功能检查另计整批墙时。双调用复核还应单列到最终代码时间及累计输入 Token，不能用第一稿 TTFT 表示复核结束。原始输出、失败原因、prompt hash、快照和测试 hash 全部保存，可离线重放。

5 已经取得的结果

v2 虽总通过数增加，但 chunks 由基线正确变为压缩错误，质量门槛失败，因此保留为负结果。v2 阈值任务的初始文字存在辅助函数归属歧义，后续明确修正并创建新 fixture 版本，不能把不同文字定义下的通过数当作同一固定测试。v3 标识符拆分一度把 `.py` 扩展名误作相关词，导致无关源码入选；v4 移除扩展名相关度后保留更小、依赖完整的上下文。

v4 为 84 次真实生成：实际总输入完整 128343、压缩 11688 Token，加权压缩 90.893%，逐任务均值 88.297%，逐任务端到端首 Token 改善均值 85.799%，会话成对请求中位首 Token 改善 89.130%。整批墙时 477.58 秒，生成总时 269.09 秒。独立重放完成 28 个 prompt hash、84 个功能结果，并重新核验模型文件。

v5 同一修正后 14 题、3 轮共 84 次生成，完整和压缩均 10 题正确，仍失败的是 CSV、旋转、区间合并和归一化。典型错误包括过滤掉非法整数、offset 为 0 时返回原对象、原地排序、仅导入校验函数而不调用。实际 Token 与 v4 相同，逐任务模型 TTFT 改善均值 88.739%，端到端均值

批次/模型	题数	完整通过	压缩通过	成对退化
最初 0.5B/native	6	3	3	0
v2 0.5B/expanded	14	5	7	1
v3 0.5B/concise	14	4	5	0
v4 1.5B/concise	14	9	9	0
v5 3B/分块预填充	14	10	10	0
v6 3B/普通复核	20	13	16	0
v7 3B/静态提示复核	20	13	16	0
v8 7B/NF4 单调用	20	17	18	0
v9 7B/NF4 首次持出	6	1	2	0
v11 7B/NF4 双调用	26	19	20	0
v12 7B/NF4 三阶段批评	26	21	20	2
v13 工作流首次单调用	4	0	0	0

表 1: 每题要求各重复均通过。不同批次曾调整提示/fixture，不能解释为纯模型规模消融。

88.476%，会话成对请求中位改善 91.037%。生成总时 342.98 秒，包含载入/warmup/测试的整批墙时 599.51 秒。这是固定小型任务上“不劣于自身完整提示”的证据，不能宣称全部代码质量达标。

v6 为 20 题、两模式、两轮、每轮初稿与复核共 160 次生成。初稿完整 12 题、压缩 14 题，复核后 13 题和 16 题；没有初稿正确变为修订错误。普通复核修正归一化两模式与压缩行解析，CSV、旋转、区间合并、价格异常传播仍失败。累计输入 Token 为完整 252648、压缩 32394，加权减少 87.178%；第一稿端到端 TTFT 逐题改善均值 86.736%，到最终代码的阶段时间均值改善 27.973%。整批墙时 1057.46 秒，调用阶段总时 694.66 秒。复核草稿及第二次上下文均计入 Token；只报第一 Token 会隐藏复核的实际工作量。

v7 仅增加公开要求及 AST 可见的静态提示：直接返回输入别名、直接修改参数、导入但未调用 helper、吞掉要求抛出的异常、跳过非法整数、删除既有导出。规则不执行代码、不看测试参数或真值，也不是正确性证明。160 次生成后完整 13 题、压缩 16 题，初稿 12 题、14 题，均与 v6 相同；因此未把这项启发式宣传为有效质量改善。累计输入完整 253622、压缩 32934，加权减少 87.015%；端到端第一稿 TTFT 改善均值 87.804%，最终代码阶段时间改善 26.066%，墙时 1035.22 秒。v6/v7 最终代码时间为各阶段求和，未包括极短的完整解码成本；后续评测改用请求开始到最终解码完成的连续墙钟，不同口径须明确区分。v7 已离线独立重建 40 个初稿提示 hash、80 个复核提示 hash，核对累计实际 Token，重放全部 160 个初稿/最终功能结果，并重新验证模型文件。

v8 在相同 20 题上换用 7B NF4、greedy、分块 1024 与单调用，共 80 次真实生成，完整 17 题、压缩 18 题，无成对退化，仍在区间合并及 helper 调用上失败。3B v6/v7 的初稿完整 12 题、压缩 14 题，复核后 13 题、16 题；7B 数值同时改变模型与权重精度，不能视为纯量化或纯规模消融。输入 Token 完整 123640、压缩 13486，加权减少 89.093%；端到端 TTFT 逐题均值改善 85.448%，最终完整解码时间改善 34.694%。调用时间 420.50 秒，整批墙时 594.32 秒。40 个提示 hash 与 80 个功能结果已独立重放，全部模型文件重新验证。

为检查过拟合于短函数与旧集合，在首次测量前固定 6 个新任务，分别为稳定拓扑排序、分层最短路、区间空隙、稀疏乘积、轮转分配及外连接。测试涵盖随机结构、重复项、空输入、非法输入和已有导出，独立参考实现通过全部 18 个测试方法，占位实现全部拒绝。v9 保持 v8 模型及单

调用配置，共 24 次生成：完整 1/6、压缩 2/6，压缩仅通过分层最短路和轮转分配。失败主要涉及公开要求中的验证、排序及重复项处理，不是隐藏 oracle 被用于模型反馈后的结果。这一批之后该集合成为开发数据，后续复核不能继续称为首次持出。输入 Token 完整 38842、压缩 6094，加权减少 84.31%；逐题压缩均值 81.82%，端到端首 Token 改善 83.62%，最终代码时间改善 34.07%。生成 285.43 秒、整批 425.57 秒；12 个提示 hash、24 个功能结果及模型文件均已独立核验。

v11 将上述 20 题与 6 题原样合并，固定快照、要求和隐藏测试，使用 7B NF4 普通复核，两轮、完整/压缩共 104 个流程、208 次生成。完整 19/26、压缩 20/26，原 20 题完整/压缩均 18/20，新 6 题仍为 1/6 与 2/6；无成对或初稿到修订退化。相对单调用，复核修正了一题完整上下文的初稿，但没有提高压缩组正确率。累计输入完整 333528、压缩 47824，加权减少 85.66%；第一稿端到端首 Token 逐题改善 85.22%，到最终完整解码时间改善 38.05%。整批墙时 2017.64 秒、调用阶段 1498.01 秒。这个相对时延改善不能掩盖第二次调用的绝对成本，也不能因全流程 Token 减少而宣称复核策略本身节省推理。v11 使用初始化依赖修复前的冻结源码，26 题修复前后依赖图一致；没有将后续修复归作该批质量收益。52 个初稿提示 hash、104 个复核提示 hash、208 个功能结果及全部模型文件已经独立离线重放核验。

v12 在相同 26 题、两模式、两轮上增加模型批评阶段：初稿、公开要求批评、修订，共 312 次计时生成。批评只见公开上下文、要求与草稿，不运行或读取隐藏测试。完整上下文 21/26、压缩 20/26，初稿仍为 18/26 与 20/26；没有初稿正确到修订错误，但归一化与稳定拓扑排序两题由完整正确变为压缩错误，质量门槛失败。相对 v11 普通复核，完整模式增加两题、压缩模式不增；相对本批初稿，完整模式增加三题。因上述成对退化，三阶段策略保留为负结果，不进入默认流程。累计输入完整 514086、压缩 86020，加权减少 83.27%；第一稿端到端首 Token 逐题改善 83.32%，最终完整解码时间改善 32.84%。生成阶段总时 3249.01 秒、整批墙时 3938.74 秒（65.65 分钟），实际最大输入 6086 Token。独立重放核验 52 个初稿、104 个批评、104 个修订提示 hash，全部 208 项功能结果和模型文件。后续受检代码允许纯函数标准库 copy/heapq；这一评分策略修订不用于追溯改写 v12 结果，重放按起始质量检查器 SHA 选择冻结模块。

v13 在任何模型测量前固定 4 个新工作流任务，模型及公开反馈协议先冻结后测试。两模式各两轮共 16 次单调用生成，完整/压缩均 0/4，故质量门槛失败；没有正确基线时不能以压缩、时延或“零退化”宣称任务成功。失败包括事务操作没有先完整验证、合并配置未深拷贝与完整验证、补丁原始坐标处理错误，以及缓存追踪的验证/状态错误；压缩缓存稿还违反纯函数 fixture 属性写入边界。输出均在 158–439 Token 内，没有耗尽 512 输出上限，不能据此将失败归于截断。生成 227.41 秒、整批 347.82 秒；输入 Token 逐题减少 75.49%，端到端首 Token 改善 78.84%、最终代码时间改善 11.22%，仅作为失败工作负载成本记录。后续 30 题公开反馈批次将 26 项既有开发题与这 4 项原样合并；4 项首次测量后成为开发数据，公开反馈算法没有在这次失败后按隐藏结果调整。

工作负载	前 (ms)	后 (ms)	范围
4096 长 SDPA 注意力	48.653	3.481	单注意力适配，5 次中位
83739 Token 治理	1046.37	192.27	带完整诊断，旧版至 v4
同一 83739 Token 治理	1046.37	73.19	v4 生产请求，无全量诊断
82700 Token 会话治理	59.67	8.71	v6 无缓存/温热缓存，11 次中位

表 2: 不同层次及工作负载分别报告，不将这些加速倍数相乘。长 Token 治理只分词/排序，不进行模型推理。

第一批 17 个治理工作负载相对冻结旧核心，带诊断中位加速 7.84 倍；v4 同套中位 10.66 倍，最小 3.75 倍。会话缓存新增 23 个工作负载逐轮验证相同选择，温热中位加速 2.64 倍，最小 1.91 倍；82700 Token 索引初始化 18.83ms、首次缓存请求 40.75ms。首题首次 Tokenizer 冷启动额外成本较高，其估计需约 23 次请求摊销，不能省略。正式 100ms 要求对应重要性分析而非任意长度完整模型推理：实际 131072 Token 的 6 种分布/切块、各 30 次共 180 次分析，最高 p95 为 53.84ms。共享机器并发负载影响分位数，128K 从未送入 32768 窗口模型。

后续增量索引通过 `PreparedContext.revised` 创建新不可变快照，只复用完全相同 Item 的词项与 Token 成本；原快照及缓存字典不共享可变状态。依赖图或节点集合变化清空闭包，计数器变更继续清空成本，最终接受完整精确计数。23 个工作负载、每个 1 块编辑并分别保持/改变依赖图，共 46 种场景各 11 轮，新索引与重新建索引在全部选择、决策或显式错误上相同。编辑至治理并含独立预算/闭包验证的中位加速 2.44 倍，最小 1.62 倍；82705 Token 保持图结构场景加速 5.47 倍。此前快照索引已温热，其初始化不在这一编辑成本内；未实现文件系统增量解析或依赖图自动更新。

TypeScript 常驻 worker 只缓存最后一份完全相同的文件/保护项快照，变更即失效，关闭进程释放缓存。8 种 32–2048 函数、1 或 8 个无关模块的工作负载各 11 轮，冷/温热的选择与闭包全部一致，温热中位加速 2.21 倍，最小 1.73 倍；整批 3.88 秒。计时包括 Python 序列化、Node IPC、索引与治理，进程启动在整批墙时内、每请求外；第一轮非缓存请求另存，使用 ByteEstimate 预算，不能称模型 Token 时延。类型精简独立采用真实 Qwen Tokenizer，8/32/128/512/2048 个无关接口的保护 Token 由 247/842/3394/14747/62934 降为 72，各 11 轮确定一致；650 预算下原 32 个以上接口案例显式拒绝，新策略得到有效上下文。失败变成功不能换算为输出时延加速；62934 只进行分词，不送模型推理。8 接口的共同成功案例端到端治理中位 3.40 降为 2.71ms；整批 3.97 秒。

TypeScript 增量编译器进一步复用上个工程中内容完全相同的 SourceFile 及 oldProgram，所有导入解析在新快照重新进行，以避免增删文件后沿用旧解析。完整源码治理结果仍重新计算，不是将旧完整分析套在已编辑工程上。26 次正反向编辑覆盖类型继承、missing 模块新增、模块初始化、语法损坏、ArkTS 回退、动态导入、全局脚本、Unicode 及保护项变化，源码/依赖/警告与冻结旧 worker 完全相同。128/512/2048/8192 声明、1/8 文件的 8 个工作负载各 11 成对轮，含 IPC、索引及治理的编辑后时延中位加速 1.34 倍，最小 1.29 倍；整批 29.85 秒。计时前旧快照构建单列，完全结果缓存在所有请求禁用；这不是模型推理或质量改善。8192 声明两种文件分布的 Node 驻留观测约 303–304MiB，旧 worker 约 298–300MiB，点采样并非 RSS 峰值，不声称内存下降。独立候选通过 5 项真实 tsc/Node 语义测试后成为默认 worker；Python 模型模块保持冻结。

注意力测试 4096 长时，native/expanded 最大分配约 2.14GiB/80.25MiB，最大绝对差 0.001953125。这是该进程 CUDA allocator 绝对峰值，含输入及用于对比的存活输出，不等于整卡占用或严格增量内存；不能仅凭数值相近省略功能质量评测。

5.1 公开样例反馈：完整批次与事后审计

v17 在既有 20+6+4 开发任务上完成两轮、双上下文、三阶段流程，360 次计时生成，另有 3 次预热。完整提示 19/30、压缩提示 20/30，初稿分别 18/30、20/30，无初稿到修订退化；按原 20/结构 6/工作流 4 分组分别为完整 18/1/0、压缩 18/2/0。前 26 题与普通复核的 19/20 相同，因此没有新增压缩质量收益。30 题早已分别测量过，不作为新的独立持出集合。

开发子组	题数	完整通过	压缩通过	有效公开样例流程/全部流程
原 20	20	18	18	0/80
结构 6	6	1	2	20/24
工作流 4	4	0	0	12/16
合计	30	19	20	32/120

120 流程中 88 次协议拒绝：48 次函数作用域、22 次参数元组要求、18 次其余解析限制。这不等于 88 次模型 Python 代码不正确。32 次有效流程在整批 v17 全部生成结束后，另对可信参考实现执行，仅 4 次全部通过，28 次在当前参考/guard/精确比较口径下不通过；这是生成样例与本地参考不一致，不自动证明公开需求本身错误。相同 case/response 的 59 项独立审计复用执行，分母仍是 120 个记录流程，不能称 59 个独立工程任务。事后结果不进入任何模型提示，也不改变已经记录的模型分数。

模型生成 4160.22 秒，其中公开样例生成 1960.73 秒；公开样例执行 60.99 秒；含加载/预热/检查墙时 4889.28 秒 (81.49 分钟)。匹配前 26 题的模型生成 3308.38 秒，普通复核同 26 题为 1498.01 秒，成本约 2.21 倍而压缩质量未增加。30 题累计实际输入 Token 为 579908/93416，加权减少 83.89%；端到端第一 Token 逐题均值减少 83.46%，最终完整代码时间减少 39.52%。这些是双上下文成对减少，不能抵消增加第三次生成的绝对成本。离线已重建 60 项初稿提示、120 项样例提示、120 项修订提示及实际 Token，重放 240 项功能结果；固定模型文件与历史 guard SHA 复核通过。保留 v17 负结果；普通单调用仍为默认。

v22 协议修订的独立完整批次。 同一 30 个开发任务预先声明一轮、双上下文、三阶段，共 180 次计时生成，另有 3 次预热。完整 19/30、压缩 20/30，初稿 18/20，无成对或初稿到修订退化，仍无压缩组正确数增益。60 流程中 51 个样例结果有效、9 个协议拒绝；协议有效率 85%，明显高于 v17 的 32/120，但有效解析不等于预期值正确，也没有带来当前集合的最终正确数改善。模型生成 2057.83 秒，公开样例执行 109.92 秒，含加载/预热/检查墙时 2553.80 秒（42.56 分钟）。累计输入 Token 加权减少 82.33%，第一稿端到端首 Token 降低率逐题均值 83.02%，到最终完整解码降低率均值 38.40%。一轮不证明轮间稳定性；不能把 v17 两轮与 v22 一轮的整批墙时直接称为加速。60 个初稿、60 个公开样例和 60 个修订提示 hash，以及 120 项功能结果和全部模型文件已离线复现。原 20 题两组均 18/20，结构 6 题完整 1/6、压缩 2/6，工作流 4 题两组均 0/4。完整/压缩模式分别 25/30 和 26/30 流程样例有效。事后审计 60 流程、56 项去重响应，耗时 151.40 秒；51 个有效流程中 14 个参考全部通过，37 个与本地参考、受限 guard 或精确比较有分歧，不能统称预期答案错误。9 项协议拒绝中 3 项无效语法、2 项字面量文本边界、2 项 JSON 起始、1 项数量边界、1 项未闭合列表。审计不修改分数、提示或候选默认，重复开发题不计新增持出任务。

6 硬件、workload 与时间成本

实际环境为 Windows 11 build 26200，Ryzen 9 8945HX（16 核/32 线程），31.3GiB RAM，RTX5070 Laptop GPU 约 7.96GiB。Python 版本 3.13.12，Transformers 版本 4.57.6，PyTorch 版本为`2.12.0.dev20260408+cu128`。模型 venv 继承已有 PyTorch，其他依赖另锁，不代表在另一台机器安装同名包即可得到相同 CUDA 内核。

模型批次	权重文件 (GB)	CPU 峰值 (GiB)	CUDA 分配峰值 (GiB)	整批墙时 (s)
v3 0.5B，56 生成	0.99	2.55	1.23	426.43
v4 1.5B，84 生成	3.09	6.46	3.42	477.58
v5 3B，84 生成	6.17	9.94	6.17	599.51
v7 3B 复核，160 生成	6.17	9.94	6.18	1035.22
v8 7B NF4，80 生成	15.23	5.71	6.21	594.32
v11 7B NF4，208 生成	15.23	5.72	6.21	2017.64
v12 7B NF4，312 生成	15.23	5.72	6.21	3938.74
v13 工作流，16 生成	15.23	5.72	6.21	347.82
v17 公开反馈，360 生成	15.23	5.72	6.21	4889.28
v22 公开协议，180 生成	15.23	5.72	6.21	2553.80

表 3: GB 为十进制磁盘大小，GiB 为二进制内存。CPU 仅本进程，CUDA 仅 allocator，不含子测试进程或整机/整卡。

只运行治理、数据生成及功能 oracle 不需要 GPU；建议 4 核以上 CPU 与 8GiB 主机内存即可开展小规模工作。这是规划建议，不是已测最小配置。真实 1.5B 评测在本机 8GiB GPU 运行，3B 需要分块预填充并已有约 6.35GiB reserved，显存余量有限。前三个模型权重及 Tokenizer 约 10.3GB，7B 再增加约 15.23GB，四个合计约 25.6GB。7B 为 Apache-2.0[26]，全部保留许可及文件锁；下载时间取决于网络。7B NF4 实测 CUDA reserved 约 6.60GiB，CPU 约 5.71GiB，通过降低 GPU 驻留权重内存可在现有卡上完成已测输入。没有同模型 FP16 载入耗时对照，磁盘权重仍为 15.23GB。

FP16 权重理论下限约为 $2P$ 字节；KV 的理想存储量为

$$B_{\mathrm{KV}} = 2\,L\,H_{\mathrm{KV}}\,d\,n \times 2 \text{ bytes.}$$

32768 Token 时，0.5B、1.5B、3B、7B 配置分别约 384、896、1152、1792MiB KV，尚未计激活、扩展 KV 中间量、workspace、缓存碎片及框架。因此不能据权重大小简单判断 8GiB 可容纳完整 32K 推理，更不能声称支持 128K。代码质量批次最大真实输入 6086 Token，100K 级数据仅用于 CPU 治理与重要性测试。

独立合成容量探测使用同一 7B NF4、1024 分块、不删输入，每个长度只运行一次，生成 13 Token。1022、4088、8184、16380 个实际输入 Token 全部完成，墙时分别 1.20、4.08、8.49、20.03 秒；CUDA allocator 绝对峰值分别 5.51、5.68、5.90、6.42GiB，reserved 峰值 5.70、5.96、6.21、6.77GiB。这些是容量观测，不是代码质量或稳定性能对照。32700 输入的阶段在上一成功检查点后 693.37 秒仍未完成，因此人工结束该容量实验，继续预定质量批次。这一时间包含下一提示构造与推理，不能称精确 kernel 时长；截止并非事前指定，必须保留该限制。终止前本进程工作集约 7.05GiB，整卡观测 7690MiB、GPU 利用率 100%；没有收到 CUDA OOM，也没有 32K 完整输出，不能将其登记为成功或分配失败。WDDM 显存压力可能参与骤慢，但未单独测量迁移，不作确定归因；该历史 v14 批次的完整输入边界为 16K。证据`model-capacity-v14.json` 保留四个完成阶段，`model-capacity-v14-interruption.json` 另存原文件 SHA、终止观测与口径。

长窗口候选的资源依据。 7B 配置为 28 层、4 个 KV 头、每头 128 维，FP16 缓存的理论载荷为

$$B_{\mathrm{KV}}(n) = 2 \times 28 \times 4 \times 128 \times 2 \times n = 57344n \text{ bytes.}$$

32700 输入对应约 1.75GiB；这是 K/V 元素载荷，不含 allocator 碎片、临时张量、权重或驱动内存。原扩展 SDPA 将 4 个 KV 头展开至 28 个查询头，单层该长度的 K/V 临时载荷约 0.437GiB。按 KV 头依次处理 7 个查询头的候选，单组展开的逻辑载荷为四分之一；单 KV 头扩展也可能保持为零 stride 视图，物化与后端 workspace 须按 allocator 测量，不能从元素数推定实际峰值。每组结束显式释放临时张量，但增加 SDPA 调用次数，并不保证吞吐收益。另一隔离候选使用 Transformers 的按层 CPU 卸载 [19]，保持 FP16 精度，将非当前层缓存移到主机并异步预取；GPU 同时存活层数、workspace 与实际峰值仍须测量。分块预填充和逐 Token 解码会反复传输增长中的缓存，不能把一次 1.75GiB 传输作为全部 PCIe 成本。两候选不修改普通窗口质量批次。分组 SDPA 随后只作独立微基准，不作为已实现的整模型收益。

CPU 卸载与分块回收的实测。 隔离 v20 用按层 FP16 缓存 CPU 卸载；v23 只在长输入卸载的每个预填充分块后同步，并调用 `empty_cache` 返回未占用 allocator 块。官方文档说明该 API 不增加 PyTorch 可用的物理显存，仅释放空闲缓存，某些情形下可缓解碎片 [20]；存活 K/V 不被删除，也不降低缓存精度。两批均在启动前限定 600 秒整个 session，含模型加载、预热、提示构造和两档探测；两个完整容量观察均在该限时内完成，原 v14 人工中止不改写为限时结果。

候选/实际输入	输出	TTFT 秒	完整生成秒	CUDA 活跃/保留 GiB	进程 RAM 峰值 GiB
v20/16380	13	18.13	19.68	6.42/6.77	5.72
v23/16380	13	18.61	20.58	6.42/6.77	5.72
v20/32688	13	281.90	319.49	5.91/11.37	10.33
v23/32688	13	50.10	54.07	5.91/6.11	7.87

16380 输入未越过 16384 阈值，两版都保持普通缓存路径；0.90 秒负值保留，不归因于回收。32688 输入完整保留，生成 13 Token，共 32701，仍在模型 32768 位置上限内。v20 CUDA 保留量超过物理 8GiB，不能将较小的活跃分配量解释为全部驻留显存；v23 控制保留量后完成该 32K 类合成容量，但不是长工程代码质量结论。WDDM 迁移/碎片或传输压力属于机制假设，没有单独驱动迁移计数，不能作确定归因。不同进程的这两次观察不作为稳定吞吐加速。

随后同一加载模型、同一 32688 Token 提示、固定 v20 后 v23 的一轮独立对照，完整 greedy 输入及输出 Token ID 逐项一致（含 13 输出，SHA 相同）；TTFT219.71/50.25 秒，完整生成 256.88/54.24 秒，CUDA 保留峰值 11.37/6.11GiB。整次会话 370.09 秒；只有一对固定顺序，不建立轮间稳定性或通用加速。该进程 RAM 高水位 12.15GiB 为累计值，不能据两条相同峰值比较两候选 RAM。另用 6 个较短合成提示、两轮交替、24 次生成核对 resident 与 v20 卸载完整 greedy IDs，全相同，耗时 129.00 秒；卸载各短提示中位均较慢，普通窗口保持 resident 默认。独立 16K/32K 进程观察与同模型配对属于容量/资源、数值兼容证据，均不换算为官方准确率。

可配置缓存接口。 v24 把隔离资源候选接成显式 `resident/offloaded/reclaimed` 策略，默认仍为 resident。非 resident 只支持 CUDA，且实际输入必须严格大于阈值（默认 16384）才卸载；reclaimed 只在显式预填充分块后同步并回收未占用 allocator 块，解码和非分块路径不额外回收。完整输入、缓存精度、原 greedy 配置和 SINGLE 默认保持不变。25 模块冻结包中相对 v22 只更改模型适配、CLI 以及公开样例的关键字模块路径拒绝；配置和 FakeGPU 生命周期 11 项通过，旧 literal 及 v22 协议 10 项检查也通过。这些是 CPU 逻辑与协议证据，实际 GPU API 验收另存，不

能从 v23 观测直接转记为 v24 结果。真实 GPU 验收随后完成：6 个短提示、两轮交替、冻结 v17d 基线及 v24 三策略，共 48 次生成，各模式完整输入/输出 Token ID 一致，涵盖非分块与 1024 分块；另一次 32688 输入/13 输出，完整提示及 greedy ID SHA 与既有 v23 相同。另有 4 次预热，不计入 48 次短提示生成；整个验收会话 308.80 秒；该长调用 TTFT51.91 秒、完整生成 57.05 秒，CUDA 活跃/保留 5.91/6.11GiB，进程 RAM 累计峰值 7.31GiB。它验证 API 集成，不是一轮新的独立性能对照，计时还包含共同的 CPU Token 捕获。验证后将 25 模块逐字推广到 `src`，保留原默认与全部冻结来源。

分组 SDPA 的微基准与取舍。 冻结 v19b 候选在首批测量前锁定 6 个 CPU 独立 dense-softmax 检查，以及 18 个 GPU 形状、每形状 5 轮交替，共 180 次计时 SDPA 调用，无模型权重加载。CPU 用 FP64 显式 QK/softmax/V 参考，最大绝对误差 1.31×10^{-6}，满足预定容差；GPU FP16 所测输入的最大绝对差均为 0，不能外推任意张量逐位一致或代码质量。微基准总墙时 12.36 秒，时延含 Python dispatch 和同步，排除结果 CPU 传输与 JSON 记录。7B 对应 28 查询头/4 KV 头、K=32700 的关键结果如下：

查询 Token	原扩展 ms	分组 ms	原增量 MiB	分组增量 MiB
1	3.96	9.35	448.01	0.009
256	13.63	12.46	465.75	18.16
1024	47.23	55.71	519.00	72.75

增量为每调用活跃 allocator 峰值减调用前分配，含输出与已跟踪 workspace，不是模型整体峰值、驱动显存或 KV 缓存总载荷。256 查询形状约快 8.6%，但当前 1024 分块形状约慢 18%，长解码形状约为原时延 2.36 倍。资源节省与吞吐成本同时成立，因此没有推广该候选，也没有为它增加整模型质量批次。若未来受更严内存约束，可研究选择性分组，但须重新验证整体模型与长输出成本。

7 继续推进的方向与工作余量

北京时间 10 月 11 日 06:30 额度剩余达到 35%，按用户指定的严格大于 35% 条件停止启动新优化试验，完成现有结果核验与报告。此前冻结了两份已知 DEV 任务的合成长工程数据：完整提示 31128/31106 Token，压缩提示 255/445 Token，保留 512 输出预算后均小于 32768。只增加无关完整函数，原公开要求、源码、测试与参考实现逐字保留；两题均无跨文件依赖。完整/所选模块的参考检查通过，占位实现被拒绝，42 个载荷文件 SHA 核对通过。准备耗时 39.14 秒、未导入 torch、零模型生成；未启动预声明的 4 次计时模型调用，因此不计长工程代码质量成果。该数据可供下一轮重复及长输出成本验证。

方向	收益依据/预计工作量	验收与测试成本
约束复核与回退	小模型常遗漏明确要求；可选二次生成或可靠公开检查。约 1–2 工作日	不使用隐藏 oracle 反馈；分列第一 Token、最终代码时间、总 Token 与初稿到修订退化。20 题两轮双调用约 160 次生成

方向	收益依据/预计工作量	验收与测试成本
检索与预算组优化	词项重叠受通用词和代码长度影响；BM25、符号检索、组成本排序。约 2–3 工作日	固定预算/保护项，新增长代码与非保护依赖召回；治理对照后再跑质量里程碑，避免每次小改重复生成
真实工程评测	当前多数任务为短纯函数；需要构建、修复、多语言与分布变化。约 3–5 工作日	首批 50–100 个任务；固定隐藏测试、工程版本及合规隔离。GPU 时间预计按实测每任务调用成本线性估算，不承诺准确总时
DevEco 与 ArkTS	TypeScript 编译器基座已完成，专用语法/SDK 和产品注入仍待适配。约 3–5 工作日	增量 worker、模板注入、完整工程构建与交互时延；需实际 SDK/IDE 及目标工程
多 session 与缓存	温热请求和内容编辑缓存已验证，实际文件系统/DevEco 事件传播与多 session 并发尚未验证。约 1–2 工作日	测事件传播、内存回收、缓存失效及并发 WAL；不把单会话温热结果外推服务吞吐
更长窗口/较强模型	7B NF4 单调用原集合 18/20、新 6 题 2/6，复杂任务与约束是质量瓶颈。约 1–2 工作日	固定新持出任务再研究复核/约束策略；长窗口另测。更高显存 GPU/远端服务为可选资源，不计入已有环境
长窗口显存治理	卸载配合分块回收已完成 32688 输入与 13 输出，完整 greedy IDs 一致；分组 SDPA 微基准有资源收益及长解码负成本，未推广。约 1–2 工作日	长工程和交替重复对照，分列 PCIe 成本、峰值和预定截止；普通短窗口卸载更慢，保留 resident 默认

上述工作量为一名开发者的工程估算，可部分并行，不能简单相加为赛事交付承诺。当前余量主要是质量提升、真实工程覆盖和集成；单次治理速度已经在已测规模上有明显收益，继续牺牲质量换极少 Token 不符合题面计分前提。

8 证据与复现

`artifacts/research-model-v4-1.5b.json` 和`research-model-v5-3b.json` 保存模型文件锁、版本、生成原文、逐题失败与实际资源；各轮冻结源码保留在 `research-vN-source/`。`research-governance-v4.json`、`research-128k-importance-v3.json` 和 `research-session-cache-v6.json` 分别对应治理、真实 128K 分析和会话缓存。`research-incremental-cache-v10.json` 记录编辑后索引复用的全部原始样本，`research-model-v8-nf4-7b.json` 为 7B 单调用质量与资源证据。`research-model-v9-holdout-nf4-7b.json` 单列首次 6 题持出表现，`initialization-map-audit.json` 记录依赖分析修复前后 26 题图一致。`research-model-v11-review-nf4-7b.json` 记录

26 题普通复核的质量负结果与成本。research-model-v12-critique-nf4-7b.json 记录三阶段批评的两项成对退化、全部原文与离线核验。research-model-v13-workflow-single-nf4-7b.json 保存首次四任务零通过及逐方法错误。research-model-v17-public-examples-nf4-7b.json、public-subgroups-v17.json 及 public-examples-reference-audit-v17.json 记录三阶段公开样例反馈、协议失败与不进入提示的参考审计。research-model-v22-public-contract-nf4-7b.json、public-subgroups-v22.json 和 public-examples-reference-audit-v22.json 保存协议修订完整批次及事后核验，报告离线定稿 SHA 为 eb8386e2bb7e539293b88e988f9618b1602e866beadc9ea2005ed2dfc8be8f0a。model-capacity-offload-v20.json、model-capacity-reclaim-v23.json 及 long-kv-token-compatibility-v23.json 分别保存独立容量观察与单对完整 Token 核对。kv-policy-api-compatibility-v24.json、v24-promotion.json 及 core-regression-v24.log 保存真实 API 验收、25 模块推广与最终回归，源码来自adapters/research-v24/；完整冻结包聚合 SHA 为 6a189ad77542a3aba4066ad56fb3fc08bb2f8b5ad9dee597aa8cb2262bff438b。grouped-attention-v19.json 保存零模型加载的分组注意力微基准及负成本，fixtures/long-engineering-v25/manifest.json 保存未进行模型测量的长工程准备数据。上述结果 JSON 均位于本项目artifacts/，数据和冻结源码路径相对本项目根目录。typescript-incremental-v18.json 与typescript-incremental-semantic-v18.json 记录编辑后编译器复用的冻结源码、成对样本与真实编译执行检查。typescript-context-v15.json 和typescript-type-import-v16.json 记录编译器缓存与类型闭包对照；selected-dependency-audit-v13.json 记录 30 个压缩后工程的参考实现执行审计。新版双调用 20 题数据为 fixtures/engineering-v5/，结果文件research-model-v6-review-3b.json 和research-model-v7-contract-3b.json 保存初稿、复核、静态提示、累计 Token 及各阶段时延；起始源码 SHA 与冻结源码一致。verify_recorded_eval 重建 prompt、核验真实 Token 并重放功能结果，不重新生成模型答案。本文件为独立 XeLaTeX 源码，无外部 bib/图片；结果按各批冻结源码和原始记录归属。

参考文献

[1] Chaspark. 赛题 11，核对日期 2026-10-10. https://www.chaspark.com/#/races/competitions/1326644835083431936.

[2] H. Jiang et al. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. EMNLP, 2023. https://aclanthology.org/2023.emnlp-main.825/.

[3] H. Jiang et al. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. ACL, 2024. https://aclanthology.org/2024.acl-long.91/.

[4] Z. Pan et al. LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. Findings of ACL, 2024. https://aclanthology.org/2024.findings-acl.57/.

[5] N. F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172, 2023. https://arxiv.org/abs/2307.03172.

[6] F. Zhang et al. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. EMNLP, 2023. https://arxiv.org/abs/2303.12570.

[7] C. D. Manning, P. Raghavan, H. Schütze. Introduction to Information Retrieval. Cambridge University Press, 2008, section 11.4. https://nlp.stanford.edu/IR-book/html/htmledition/okapi-bm25-a-non-binary-model-1.html.

[8] J. Ainslie et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. EMNLP, 2023. https://aclanthology.org/2023.emnlp-main.298/.

[9] T. Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS, 2022. https://arxiv.org/abs/2205.14135.

[10] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP, 2023. https://arxiv.org/abs/2309.06180.

[11] A. Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS, 2023. https://arxiv.org/abs/2303.17651.

[12] B. Chen et al. CodeT: Code Generation with Generated Tests. ICLR, 2023. https://arxiv.org/abs/2207.10397.

[13] X. Chen, M. Lin, N. Schärli, D. Zhou. Teaching Large Language Models to Self-Debug. ICLR, 2024. https://arxiv.org/abs/2304.05128.

[14] Microsoft. Using the Compiler API, TypeScript wiki. 访问日期 2026-10-11；文档对应 6 及以前的 API。https://github.com/microsoft/TypeScript/wiki/Using-the-Compiler-API.

[15] Microsoft. TypeScript Modules Reference: type-only imports and exports. https://www.typescriptlang.org/docs/handbook/modules/reference.html.

[16] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. https://www.swebench.com/SWE-bench/.

[17] PyTorch. Scaled Dot Product Attention tutorial. 访问日期 2026-10-11. https://docs.pytorch.org/tutorials/intermediate/scaled_dot_product_attention_tutorial.html.

[18] Hugging Face. Transformers Attention Interface, v4.57.1. 实装版本 4.57.6. https://huggingface.co/docs/transformers/v4.57.1/attention_interface.

[19] Hugging Face. KV cache strategies, v4.57.1；CPU offloading 与缓存容量取舍。实装版本 4.57.6，访问日期 2026-10-11. https://huggingface.co/docs/transformers/v4.57.1/kv_cache.

[20] PyTorch. `torch.cuda.memory.empty_cache`, 在线文档 2.14，访问日期 2026-10-11；本机执行版本 2.12.0.dev20260408+cu128。https://docs.pytorch.org/docs/2.14/generated/torch.cuda.memory.empty_cache.html.

[21] Qwen. Qwen2.5-Coder-0.5B-Instruct model card. https://huggingface.co/Qwen/Qwen2.5-Coder-0.5B-Instruct.

[22] Qwen. Qwen2.5-Coder-1.5B-Instruct model card. https://huggingface.co/Qwen/Qwen2.5-Coder-1.5B-Instruct.

[23] Qwen. Qwen2.5-Coder-3B-Instruct model card and license. https://huggingface.co/Qwen/Qwen2.5-Coder-3B-Instruct.

[24] Hugging Face. bitsandbytes installation, 访问日期 2026-10-11. https://huggingface.co/docs/bitsandbytes/main/en/installation.

[25] Hugging Face. Transformers bitsandbytes quantization, v4.57.1. https://huggingface.co/docs/transformers/v4.57.1/quantization/bitsandbytes.

[26] Qwen. Qwen2.5-Coder-7B-Instruct model card, Apache-2.0. https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct.

[27] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS, 2023. https://arxiv.org/abs/2305.14314.